

Gestão de Dados: warehouse, lakehouse, data mesh e OLAP

Data Warehouse: A Base da Inteligência de Negócios

Um Data Warehouse é muito mais do que um simples repositório de dados. É uma arquitetura sofisticada que integra informações de múltiplas fontes, transformando dados brutos em insights estratégicos que impulsionam decisões empresariais inteligentes.

Esta tecnologia revolucionou a forma como as organizações gerenciam e analisam suas informações, permitindo análises históricas profundas e projeções futuras baseadas em dados consolidados e confiáveis.

O Que É um Data Warehouse?

"Um data warehouse é uma coleção de dados orientada por assunto, integrada, variante no tempo e não volátil, para apoiar o processo de tomada de decisão da gerência." — W. H. Inmon

Embora existam diversas definições, o conceito central permanece consistente: um Data Warehouse é um banco de dados especializado para suporte à decisão, mantido separadamente dos sistemas operacionais da organização. Ele fornece uma plataforma sólida de dados históricos consolidados, especificamente projetada para análise e processamento de informações estratégicas.

O processo de **data warehousing** envolve a construção, manutenção e utilização efetiva desses repositórios, permitindo que organizações transformem dados em vantagens competitivas concretas.

Orientado por Assunto

Cliente

Dados demográficos, histórico de compras, preferências e comportamento de consumo

Produto

Catálogo completo, especificações, desempenho de vendas e ciclo de vida

Vendas

Transações, receitas, tendências de mercado e análise de performance

Um Data Warehouse é organizado em torno de temas principais do negócio, não de processos operacionais diários. Essa abordagem permite focar na modelagem e análise de dados para tomadores de decisão, proporcionando uma visão simples e concisa de questões específicas.

Ao contrário dos sistemas transacionais, o Data Warehouse exclui dados que não são úteis no processo de suporte à decisão, mantendo apenas informações relevantes e estratégicas para análises de negócios.

Integrado: Unificando Fontes Heterogêneas

A construção de um Data Warehouse exige a integração de múltiplas fontes de dados heterogêneas, incluindo bancos de dados relacionais, arquivos planos e registros de transações online. Este processo complexo garante que informações dispersas sejam unificadas em um repositório coerente.

Aplicação de Técnicas de Limpeza

Identificação e correção de inconsistências, erros e duplicações nos dados originais

Padronização de Convenções

Garantia de consistência em nomenclaturas, estruturas de codificação e medidas de atributos

Transformação e Conversão

Conversão de dados para o formato do warehouse durante o processo de migração

❏ **Exemplo prático:** Preços de hotéis podem variar entre fontes de dados devido a diferenças em moeda, impostos inclusos, café da manhã coberto e estacionamento. O processo de integração resolve essas inconsistências.

Variante no Tempo: Perspectiva Histórica

O horizonte temporal de um Data Warehouse é significativamente mais longo do que o dos sistemas operacionais, proporcionando uma dimensão histórica crucial para análises estratégicas.

Bancos de Dados Operacionais

- Mantêm apenas dados atuais e recentes
- Focam em transações do momento presente
- Otimizados para operações em tempo real

Data Warehouse

- Armazena informações históricas de 5 a 10 anos
- Permite análise de tendências de longo prazo
- Suporta previsões baseadas em padrões históricos

Toda estrutura de chave no Data Warehouse contém um elemento de tempo, seja explícita ou implicitamente. Esta característica permite rastreamento de mudanças ao longo do tempo e análises temporais sofisticadas, algo que nem sempre está presente nos dados operacionais.

Não Volátil: Estabilidade e Consistência

Independência Física

O Data Warehouse é um repositório fisicamente separado do ambiente operacional, contendo dados transformados e otimizados especificamente para análise. Esta separação garante que as operações analíticas não interfiram com o desempenho dos sistemas transacionais.

Natureza Estática

Atualizações operacionais não ocorrem no ambiente do Data Warehouse. Uma vez que os dados são carregados, eles permanecem inalterados, garantindo consistência nas análises e relatórios ao longo do tempo. Esta estabilidade é fundamental para auditorias e comparações históricas.

Operações Simplificadas

O ambiente não requer mecanismos complexos de processamento de transações, recuperação ou controle de concorrência. Apenas duas operações são necessárias: carga inicial dos dados e acesso para consultas analíticas, simplificando significativamente a arquitetura.

OLTP vs. OLAP e a Necessidade de um DW Separado

OLTP – Online Transactional Processing

Focado em operações do SGBD, consultas rápidas e processamento de transações em tempo real para operações diárias do negócio.

- Otimizado para inserções, atualizações e exclusões
- Dados normalizados para eficiência operacional
- Resposta em milissegundos
- Suporta operações concorrentes massivas

OLAP – Online Analytical Processing

Projetado para operações de Data Warehouse, com análises multidimensionais como drilling, slicing e dicing para insights estratégicos.

- Otimizado para consultas complexas e agregações
- Dados desnormalizados para análise rápida
- Resposta em segundos ou minutos
- Suporta análises históricas e preditivas

Por Que um Data Warehouse Separado?

A separação entre sistemas operacionais e analíticos não é apenas uma preferência arquitetural, mas uma **necessidade estratégica**.

Alto Desempenho para Ambos

SGBD otimizado para OLTP com indexação e controle de concorrência específicos.
Warehouse ajustado para OLAP com suporte a consultas complexas e consolidação de dados.

Funções e Dados Diferentes

Suporte à decisão requer dados históricos que bancos operacionais normalmente não mantêm. Necessidade de consolidação via agregação de fontes heterogêneas.

Qualidade e Consistência

Fontes distintas usam representações e formatos inconsistentes. O DW garante padrões unificados de qualidade através da reconciliação de dados.

❗ Existem cada vez mais sistemas que realizam análises OLAP diretamente em bancos de dados relacionais, mas a arquitetura separada ainda oferece vantagens significativas para cenários empresariais complexos.

Arquitetura e Modelos de Data Warehouse

Arquitetura em 3 Camadas

Camada Inferior: Servidor do Data Warehouse

Fundação da arquitetura onde os dados são armazenados, gerenciados e organizados. Inclui o banco de dados do warehouse e os servidores que o suportam, garantindo armazenamento eficiente e acesso otimizado.

Camada Intermediária: Servidor OLAP

Camada de processamento analítico que transforma dados brutos em insights acionáveis. Implementa o motor de análise multidimensional, permitindo operações como drill-down, roll-up, slice e dice.

Camada Superior: Ferramentas Front-End

Interface com o usuário final através de ferramentas de consulta, relatórios, análise, visualização e mineração de dados. Torna os insights acessíveis para tomadores de decisão através de dashboards e relatórios.

Esta arquitetura garante separação de responsabilidades, escalabilidade e flexibilidade.

Três Modelos de Data Warehouse

Enterprise Warehouse

Coleta todas as informações sobre assuntos que abrangem toda a organização, fornecendo uma visão unificada e abrangente do negócio completo. É a solução mais robusta e complexa.

Data Mart

Um subconjunto de dados corporativos valiosos para grupos específicos de usuários. Escopo limitado a áreas selecionadas (ex: data mart de marketing). Pode ser independente ou dependente do warehouse corporativo.

Virtual Warehouse

Um conjunto de visões sobre bancos de dados operacionais. Apenas algumas das possíveis visões resumidas são materializadas, oferecendo uma alternativa leve e ágil, embora com limitações de desempenho.

A escolha do modelo depende de tamanho da organização, complexidade analítica, orçamento e urgência. Muitas organizações começam com Data Marts e evoluem para um Enterprise Warehouse.

Extração, Transformação e Carga (ETL)

Extração

Obtenção de dados de múltiplas fontes heterogêneas e externas

Limpeza

Detecção de erros e correção quando possível

Transformação

Conversão do formato legado para formato do warehouse

Carga

Ordenação, consolidação e construção de índices

Atualização

Propagação de updates das fontes para o warehouse

Processo ETL — Extração, Transformação e Carga

Referência: Inmon (2002), Kimball & Ross (2013)

ENTRADA: fontes[] (BD relacionais, arquivos, APIs), destino (Data Warehouse)

PASSO 1 — Extração

PARA CADA fonte em fontes[]:

dados_brutos ← extrair(fonte)

registrar_metadados(fonte, timestamp)

PASSO 2 — Limpeza

PARA CADA registro em dados_brutos:

SE contém_erro(registro):

SE corrigível → corrigir(registro)

SENÃO → descartar(registro) e registrar_log()

PASSO 3 — Transformação

PARA CADA registro limpo:

registro ← converter_formato(registro, esquema_DW)

registro ← padronizar_codigos(registro)

registro ← aplicar_regras_negocio(registro)

PASSO 4 — Carga

ordenar(dados_transformados)

consolidar_duplicatas(dados_transformados)

construir_indices_e_particoes(destino)

carregar(dados_transformados → destino)

PASSO 5 — Atualização (incremental)

ENQUANTO fonte_atualizada():

delta ← capturar_mudancas(fonte)

propagar(delta → destino)

SAÍDA: Data Warehouse atualizado e consistente

Repositório de Metadados: O Catálogo do Warehouse

Metadados são os dados que definem os objetos do warehouse — um catálogo essencial para governança, manutenção e uso efetivo do sistema.

Estrutura do Warehouse

- Schema, visões e dimensões
- Hierarquias e dados derivados
- Localização e conteúdo dos data marts

Algoritmos e Mapeamentos

- Algoritmos usados para sumarização
- Mapeamento do ambiente operacional
- Transformações aplicadas aos dados

Metadados Operacionais

- Linhagem de dados e histórico de transformações
- Atualidade dos dados (ativo, arquivado ou purgado)
- Estatísticas de uso e relatórios de auditoria

Performance e Negócios

- Dados relacionados ao desempenho do sistema
- Termos e definições de negócios
- Propriedade de dados e políticas de cobrança

Data Lake: Repositório Centralizado de Dados

Um data lake é um repositório centralizado que armazena todos os dados estruturados e não estruturados de uma organização, em qualquer escala. Esta abordagem revolucionária permite que as empresas mantenham dados em seu formato bruto, sem a necessidade de estruturação prévia.

Os dados são armazenados em seu estado original, permitindo a execução de diferentes tipos de análises — desde dashboards e visualizações até processamento de big data, análises em tempo real e machine learning. Esta flexibilidade orienta decisões empresariais mais informadas e estratégicas.

A capacidade de armazenar dados diversos sem transformação inicial reduz significativamente o tempo de ingestão e permite que as organizações preservem a granularidade e riqueza dos dados originais.

Armazenamento Flexível

Dados em formato bruto

Análises Diversas

Múltiplas aplicações

Escalabilidade

Qualquer volume

Arquitetura Conceitual de Data Lakes

A arquitetura de um data lake organiza o fluxo de dados desde a ingestão até o consumo analítico, garantindo governança e qualidade em todas as camadas do sistema.

Organização de Dados Empresariais

Dados Não Estruturados

Documentos, imagens, vídeos e outros formatos livres

Extração

Processamento e parsing de dados brutos

Dados com Schema

Informações estruturadas e organizadas

Dados Estruturados

Tabelas relacionais prontas para análise

Analytics

Insights e visualizações de negócio

O data lake suporta todo este fluxo, desde a ingestão de dados brutos até a geração de análises sofisticadas que impulsionam decisões estratégicas.

Desafios na Implementação de Data Lakes

Volume Massivo

Centenas de milhares de conjuntos de dados ou mais, exigindo gerenciamento sofisticado

Consultas Complexas

Queries por palavra-chave, descoberta de datasets relacionáveis, identificação de features relevantes

Gestão de Metadados

Tarefa crítica na construção de data lakes eficientes e governados

O sucesso de um data lake depende fundamentalmente da capacidade de gerenciar metadados de forma eficaz, permitindo descoberta, governança e qualidade de dados em escala empresarial.

Tarefas e Integração na Construção de Data Lakes

7 Tarefas Fundamentais

Ingestão

Captura e carregamento de dados de múltiplas fontes

Gestão de Metadados

Catalogação e governança de ativos de dados

Integração

Combinação de datasets relacionados

Versionamento

Controle de evolução e histórico de dados

Estas sete tarefas formam o núcleo operacional de qualquer implementação bem-sucedida de data lake.

Extração

Inferência de tipos e estruturação de dados brutos

Limpeza

Tratamento de inconsistências e qualidade

Descoberta

Localização de dados relevantes

Desafios de Integração

Os formatos de datasets no mundo real podem ser altamente heterogêneos, apresentando desafios únicos de integração e processamento.

Ingestão e Extração

Processo de trazer datasets estruturados para o data lake, incluindo ingestão de dados já estruturados e extração de dados estruturados de fontes não estruturadas e semi-estruturadas.

Integração de Dados

Tarefa de encontrar tabelas que podem ser unidas (join) ou combinadas (union), ou de popular esquemas sob demanda com todos os dados do lake que conformam ao schema definido.

 **Exemplo Prático:** Enriquecer Registros Eletrônicos de Saúde (EPR) usando dados de diversos conjuntos de dados pessoais de saúde não padronizados para melhor prever riscos à saúde. Isso envolve unir ou combinar tabelas de diferentes datasets e fontes heterogêneas.

Gestão de Metadados: Estratégias Avançadas

Enriquecimento Semântico

Enriquecer dados e metadados com informação semântica, suportando consultas baseadas em templates sobre metadados

Extração Profunda

Extrair metadados profundamente embutidos e metadados contextuais para suportar descoberta baseada em tópicos

Integração Visual

Integrar metadados sobre dados, usuários e consultas, suportando análises visuais e interativas

A gestão eficaz de metadados transforma um data lake de um repositório passivo em um ativo inteligente que facilita descoberta, governança e extração de valor dos dados.

Dois Propósitos e Ecossistemas Distintos

Data Warehouses

Casos de Uso Analíticos

Suporte à decisão baseado em dados estruturados

- Dados estruturados e organizados
- SQL e Python como linguagens principais
- Foco em relatórios e BI tradicional
- Schemas definidos e otimizados

Data Lakes

Casos de Uso Operacionais

Inteligência de dados em aplicações

- Dados brutos e diversos formatos
- Java, Scala, Python, R e SQL
- Machine learning e analytics avançados
- Flexibilidade e experimentação

Convergência Emergente: As fronteiras entre data warehouses e data lakes estão se tornando cada vez mais fluidas, com arquiteturas modernas buscando combinar os melhores aspectos de ambas as abordagens.

Data Lakehouses: O Melhor de Dois Mundos

"Um data lakehouse é uma nova arquitetura aberta de gerenciamento de dados que combina a flexibilidade, custo-eficiência e escala de data lakes com o gerenciamento de dados e transações ACID de data warehouses, permitindo business intelligence (BI) e machine learning (ML) sobre todos os dados."

— *Databricks*

O conceito de data lakehouse representa uma evolução significativa na arquitetura de dados, eliminando a necessidade de manter sistemas separados para diferentes cargas de trabalho.

Esta abordagem unificada reduz a complexidade operacional, minimiza a duplicação de dados e permite que as organizações obtenham insights mais rapidamente, mantendo a governança e qualidade necessárias para decisões críticas de negócio.

1

Arquitetura Unificada

Sistema único para múltiplas cargas

50%

Redução de Custos

Menos duplicação de dados

Características Fundamentais dos Lakehouses

Suporte a Transações

Garantias ACID para integridade de dados

Governança de Schema

Enforcement de estrutura e qualidade

Suporte a BI

Performance otimizada para análises

Desacoplamento Storage/Compute

Escalabilidade independente de recursos

Formatos Abertos

Interoperabilidade e evita lock-in

Tipos de Dados Diversos

De não estruturado a estruturado

Cargas de Trabalho Diversas

- Data science e experimentação
- Machine learning em produção
- SQL e analytics tradicionais
- Processamento de lotes e streams

Streaming End-to-End

Capacidade de processar dados em tempo real e gerar relatórios instantâneos, eliminando latências tradicionais entre ingestão e análise.

Data Fabric e Virtualização de Dados

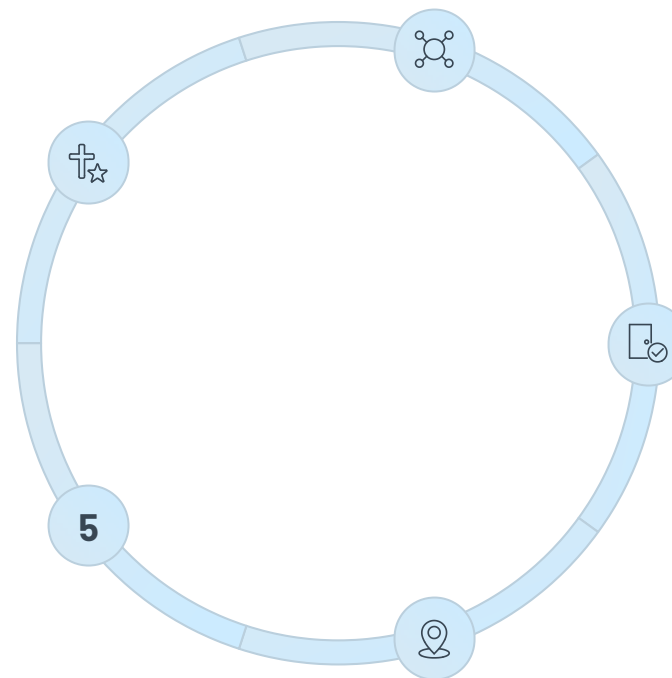
"Data fabric é uma arquitetura que facilita a integração end-to-end de vários pipelines de dados e ambientes de nuvem através do uso de sistemas inteligentes e automatizados"

— IBM

Arquitetura Holística

Gerenciamento unificado de dados em todo o ecossistema

Um data fabric fornece uma camada de abstração inteligente que permite às organizações tratar dados distribuídos como se fossem um único repositório lógico, simplificando significativamente a governança e o consumo de dados.



Abstração de Complexidade

Ocultar heterogeneidade subjacente

Ponto Único de Acesso

Acesso consolidado a todos os dados

Independência de Localização

Dados acessíveis independentemente de fonte, formato ou tecnologia

Data Mesh: Arquitetura Descentralizada

Um data mesh cria múltiplos sistemas específicos por domínio, cada um especializado de acordo com suas funções e usos, trazendo assim os dados para mais perto dos consumidores.

Esta abordagem fundamentalmente diferente trata dados como um produto, com equipes de domínio assumindo propriedade completa de seus dados - desde a qualidade até a entrega.



Propriedade por Domínio

Equipes especializadas gerenciam seus próprios dados como produtos

Arquitetura Federada

Governança distribuída com padrões compartilhados

Self-Service Data Platform

Infraestrutura que habilita autonomia dos domínios

Data Mesh é um padrão arquitetural específico focado no gerenciamento de dados, enfatizando descentralização, ownership de domínio, e tratamento de dados como produtos de primeira classe.

Virtualização de Dados: Componente Essencial

"Virtualização de dados é uma das tecnologias que habilita uma abordagem de data fabric. Em vez de mover fisicamente os dados de várias fontes on-premises e na nuvem usando os processos tradicionais de ETL (extract, transform, load), uma ferramenta de virtualização de dados conecta-se às diferentes fontes, integrando apenas os metadados necessários e criando uma camada virtual de dados"

— *Definição Técnica*

Benefícios da Virtualização

- Elimina necessidade de movimentação física de dados
- Reduz latência no acesso a informações
- Diminui custos de armazenamento e ETL
- Fornece visão unificada em tempo real
- Simplifica manutenção e governança

Integração Arquitetural

A virtualização de dados pode ser um componente ou camada em uma arquitetura de data fabric, fornecendo a abstração necessária para acessar dados distribuídos sem replicação física.

Esta tecnologia é fundamental para organizações que precisam de acesso ágil a dados distribuídos mantendo fonte única de verdade.

Das Tabelas e Planilhas aos Cubos de Dados

Um data warehouse é fundamentado em um modelo de dados multidimensional que visualiza informações na forma de um cubo de dados. Esta estrutura revolucionária permite análises complexas e eficientes de grandes volumes de dados empresariais.

Um cubo de dados, como o de vendas, possibilita modelar e visualizar dados em múltiplas dimensões simultaneamente. Por exemplo, você pode analisar vendas por produto, região e período temporal ao mesmo tempo.

Componentes Principais

- **Tabelas de dimensão:** item (nome, marca, tipo) ou tempo (dia, semana, mês, trimestre, ano)
- **Tabela fato:** contém medidas (como valor_vendido) e chaves para cada tabela de dimensão relacionada
- **Cuboides:** formam a estrutura em rede do cubo de dados

Base Cuboid

O cubo base n-dimensional que contém o nível mais detalhado de dados

Apex Cuboid

O cuboide 0-dimensional no topo, com o mais alto nível de sumarização

Lattice de Cuboides

A estrutura completa que forma o cubo de dados multidimensional

Modelagem Conceitual de Data Warehouses

A modelagem dimensional é o coração do data warehouse, definindo como dimensões e medidas se organizam para suportar análises complexas. Três esquemas principais dominam esta arquitetura, cada um com características e aplicações específicas.

Star Schema

Uma tabela fato central conectada a um conjunto de tabelas de dimensão, formando uma estrutura em estrela simples e eficiente

- Estrutura desnormalizada
- Consultas mais rápidas
- Fácil compreensão

Snowflake Schema

Um refinamento do star schema onde hierarquias dimensionais são normalizadas em tabelas menores, criando forma similar a um floco de neve

- Estrutura normalizada
- Economia de espaço
- Menor redundância

Fact Constellation

Múltiplas tabelas fato compartilham tabelas de dimensão, visto como coleção de estrelas (também chamado galaxy schema)

- Múltiplos processos de negócio
- Dimensões compartilhadas
- Análises integradas

Star Schema: Um Exemplo Detalhado

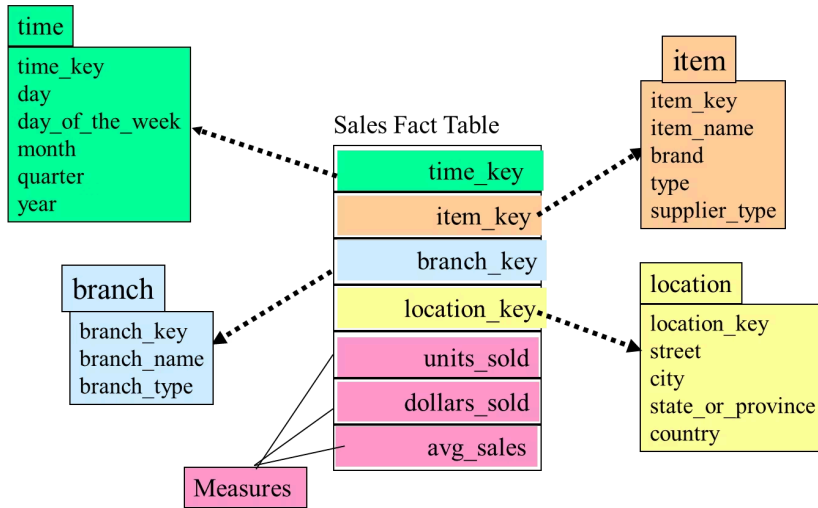


Tabela Fato de Vendas

A tabela central que armazena as métricas de negócio e as chaves estrangeiras para as dimensões.

Chaves de Dimensão:

- time_key (chave temporal)
- item_key (chave de produto)
- branch_key (chave de filial)
- location_key (chave de localização)

Medidas:

- units_sold (unidades vendidas)
- dollars_sold (valor vendido)
- avg_sales (média de vendas)

📌 **Vantagem Principal:** No star schema, cada dimensão está diretamente conectada à tabela fato, resultando em consultas SQL simples com menos JOINS e melhor performance em queries analíticas.

Snowflake Schema: Estrutura Normalizada

O snowflake schema representa uma evolução do star schema através da normalização das tabelas de dimensão. Neste modelo, as hierarquias dentro das dimensões são decompostas em múltiplas tabelas relacionadas, criando uma estrutura ramificada que lembra um floco de neve.

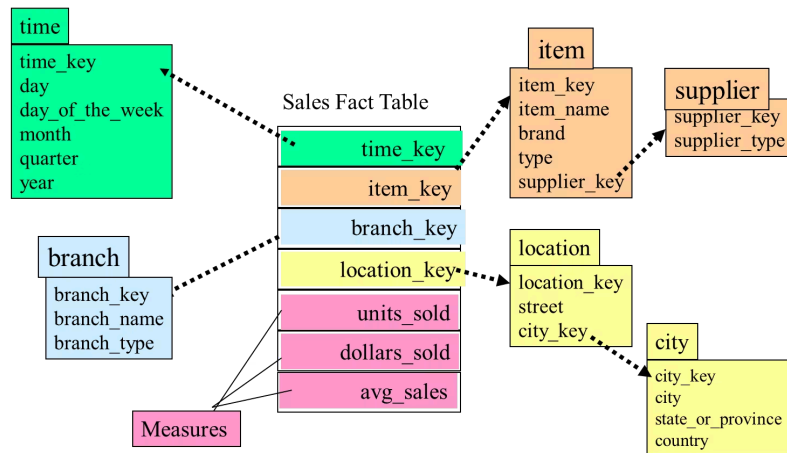


Tabela Fato Central

Mantém as mesmas chaves e medidas: time_key, item_key, branch_key, location_key, units_sold, dollars_sold, avg_sales

Dimensões Normalizadas

Cada dimensão é decomposta em hierarquias, com tabelas separadas para cada nível (ex: item → categoria → departamento)

Relacionamentos em Cascata

As tabelas de dimensão conectam-se entre si, formando cadeias de relacionamentos que refletem a hierarquia natural dos dados

Vantagens

- Menor redundância de dados
- Economia de espaço de armazenamento
- Facilita manutenção de hierarquias
- Integridade referencial mais forte

Desvantagens

- Consultas mais complexas
- Maior número de JOINS necessários
- Performance pode ser menor em queries
- Mais difícil para usuários entenderem

Fact Constellation: Múltiplas Perspectivas de Análise

O fact constellation, também conhecido como galaxy schema, representa o modelo mais complexo e flexível de data warehouse. Ele permite que múltiplas tabelas fato compartilhem dimensões comuns, possibilitando análises integradas de diferentes processos de negócio.

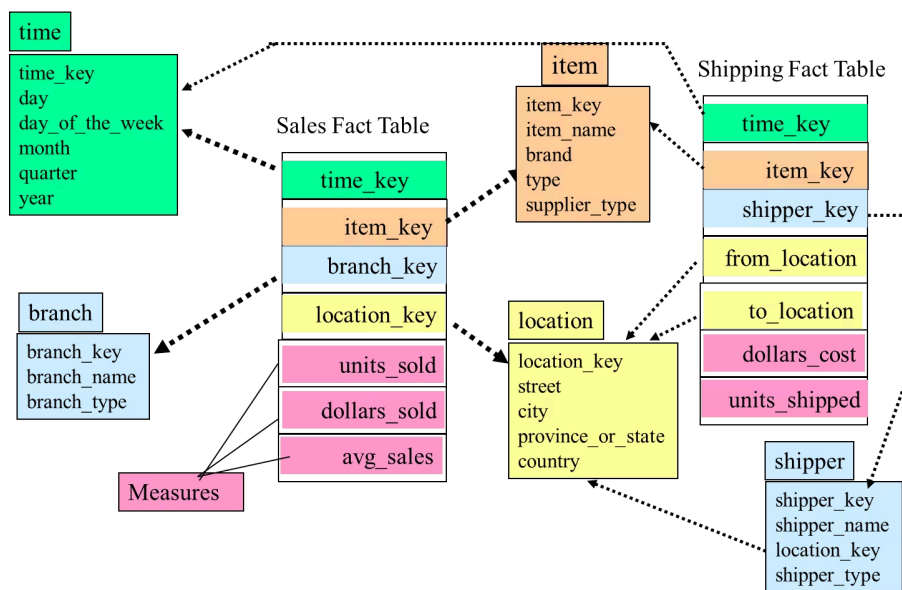


Tabela Fato de Vendas

Chaves:

- time_key
- item_key
- branch_key
- location_key

Medidas:

- units_sold
- dollars_sold
- avg_sales

Tabela Fato de Envios

Chaves:

- time_key
- item_key
- shipper_key
- from_location
- to_location

Medidas:

- dollars_cost
- units_shipped

📌 **Aplicação Prática:** Este modelo é ideal quando você precisa analisar múltiplos processos de negócio que compartilham dimensões comuns, como vendas e envios que compartilham as dimensões de tempo, item e localização.

Hierarquia de Conceitos para Dimensões

As dimensões em um data warehouse não são planas - elas contêm hierarquias naturais que permitem análises em diferentes níveis de granularidade. Compreender estas hierarquias é essencial para realizar operações de drill-down e roll-up eficazmente.

Exemplo: Hierarquia da Dimensão Localização

All (Todos)

Nível mais agregado - visão global de todos os dados

Region (Região)

Europa, América do Norte, Ásia, etc.

Country (País)

Alemanha, Espanha, Canadá, México, etc.

City (Cidade)

Frankfurt, Vancouver, Toronto, etc.

Office (Escritório)

Nível mais detalhado - L. Chan, M. Wind, etc.

Categorias de Medidas em Cubos de Dados

As medidas em cubos de dados se classificam em três categorias com propriedades matemáticas distintas que afetam a pré-computação.

Distributivas

- `count()`
- `sum()`
- `min()`
- `max()`

Mais eficientes para pré-computação.

Algébricas

- $\text{avg}(x) = \text{sum}(x) / \text{count}(x)$
- `standard_deviation()`
- `variance()`

Calculadas a partir de funções distributivas.

Holísticas

- `median()`
- `mode()`
- `rank()`

Exigem dados completos — pré-agregação inviável.

Fundamentos Conceituais do OLAP

Online Analytical Processing (OLAP) representa a tecnologia central que permite análises multidimensionais eficientes em data warehouses. É a ponte entre dados armazenados e insights de negócio acionáveis.

Visão Multidimensional

OLAP permite visualizar dados através de múltiplas dimensões simultaneamente, facilitando a compreensão de relacionamentos complexos e padrões ocultos nos dados empresariais.

Análise Rápida

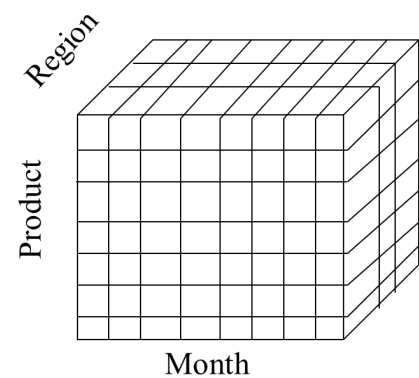
Estruturas otimizadas e pré-agregações permitem respostas em tempo quase real para consultas complexas envolvendo milhões de registros.

Navegação Intuitiva

Operações como drill-down, roll-up, slice e dice permitem que analistas explorem dados naturalmente, seguindo o raciocínio analítico.

Dados Multidimensionais em Ação

Considere o volume de vendas como uma função de produto, mês e região. Este exemplo ilustra como múltiplas dimensões se combinam para criar uma estrutura analítica poderosa e flexível.



Dimensões: Produto, Localização, Tempo

Cada dimensão possui hierarquias naturais que permitem agregações em diferentes níveis de detalhe.

Produto

Industry → Category → Product

Do nível mais agregado (indústria) até produtos individuais

Localização

Region → Country → City → Office

Da visão regional até escritórios específicos

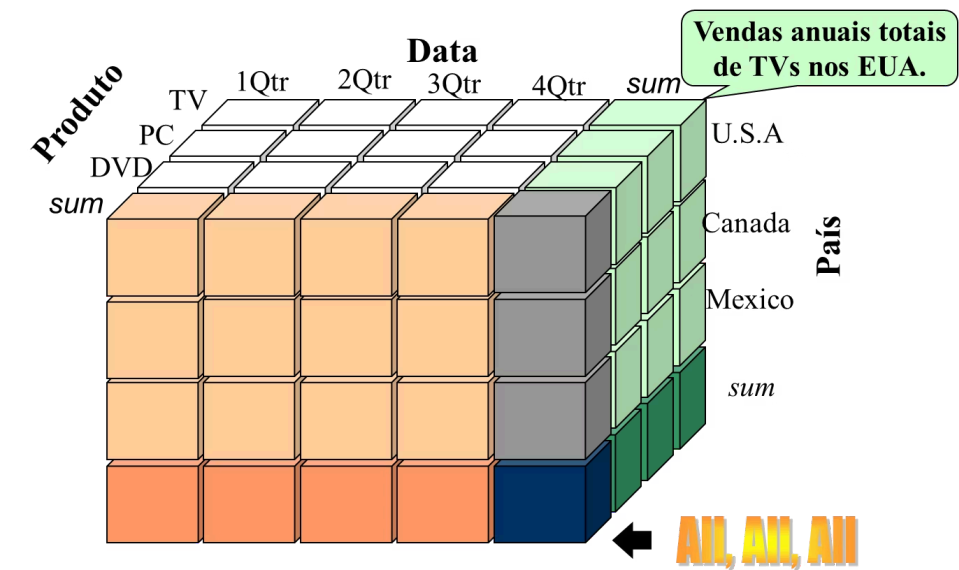
Tempo

Year → Quarter → Month → Week → Day

De anos completos até dias individuais

📌 **Exemplo Prático:** Um gerente pode iniciar analisando vendas anuais por região, fazer drill-down para vendas mensais em países específicos, e finalmente examinar vendas diárias de produtos individuais em escritórios particulares.

Exemplo de Cubo de Dados



Este exemplo demonstra um ponto específico no cubo de dados tridimensional, representando as vendas totais anuais de televisores nos EUA. É uma agregação que combina:

- **Dimensão Produto:** TV (categoria específica)
- **Dimensão Localização:** U.S.A. (país específico)
- **Dimensão Tempo:** Year (nível anual)

Esta célula do cubo contém a medida agregada (vendas totais) para esta combinação específica de dimensões.

Seleção de Dimensões

Escolha dos atributos relevantes para análise (produto, local, tempo)

Definição de Granularidade

Seleção do nível de detalhe para cada dimensão (TV, USA, Year)

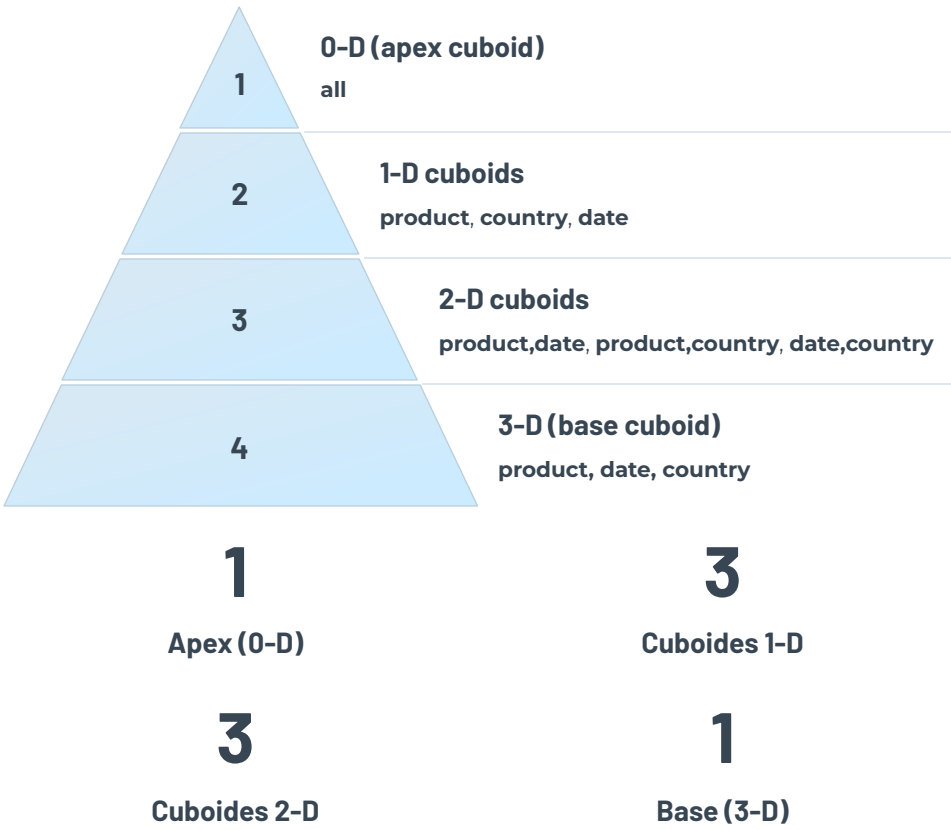
Agregação de Medidas

Cálculo das métricas de negócio (vendas totais anuais)

Data Cube: Cuboids e Lattice

Um cubo de dados completo é composto por múltiplos cuboides organizados em uma **lattice hierárquica**, cada um representando uma combinação diferente de dimensões — permitindo agregações eficientes em todos os níveis.

Estrutura de Cuboids



Conceitos da Lattice

Base vs. Células Agregadas

Células base contêm dados granulares; **células agregadas** armazenam sumarizações pré-computadas.

Ancestrais vs. Descendentes

Relações de agregação entre níveis da lattice — cuboides de menor dimensão são ancestrais dos de maior dimensão.

Pais vs. Filhos

Conexões diretas entre níveis adjacentes da hierarquia de cuboids.

Apex vs. Base Cuboid

Apex: uma célula com agregação total (top-down).
Base: máximo detalhe, ponto de partida para roll-up.

Exemplo de Navegação – Cubo de Vendas (*data, produto, localização, loja*)

Célula	Descrição
(*, *, *, *)	Agregação total — topo do lattice
(*, milk, *, *)	Todas as vendas de leite
(*, milk, Urbana, *)	Leite em Urbana
(*, milk, Chicago, *)	Leite em Chicago
(9/15, milk, Urbana, *)	Leite em Urbana no dia 9/15
(9/15, milk, Urbana, Dairy_land)	Célula base mais granular

Online Analytical Processing (OLAP)

OLAP representa a tecnologia que permite explorar sistematicamente todos os possíveis subespaços de um cubo de dados em busca de padrões interessantes e insights valiosos para o negócio.

Exploração Conceitual

Conceitualmente, podemos explorar todos os possíveis subespaços de dados em busca de padrões interessantes, mas surgem questões fundamentais sobre eficiência e relevância.

Problemas Fundamentais

Quais padrões são interessantes? Como definir relevância e valor em meio a milhares de possíveis agregações?

Como explorar sistematicamente? Métodos eficientes para navegar todos os subespaços sem computação excessiva.

Importância de Agregações

Agregações e group-bys são ferramentas essenciais em análise e sumarização de dados. No benchmark TPC, das 83 queries padrão, agregações aparecem 59 vezes e group-bys 20 vezes.

```
SELECT time, altitude, AVG(temp)
FROM weather
GROUP BY time, altitude;
```

OLAP fornece as técnicas que respondem queries analíticas multidimensionais (MDA) de forma eficiente, permitindo que analistas de negócio obtenham insights rapidamente sem necessidade de conhecimento técnico profundo em SQL ou estruturas de banco de dados.

Operações OLAP

As operações OLAP permitem que analistas naveguem pelos dados em diferentes níveis de granularidade, explorando relacionamentos e descobrindo insights através de transformações sistemáticas do cubo de dados.

Roll-Up (Drill-Up)

Sumariza dados subindo na hierarquia ou através de redução dimensional.

(Day, Store, Product type) → (Month, City, *)

Drill-Down (Roll-Down)

Reverso do roll-up: desce para dados mais detalhados ou introduz novas dimensões.

- Month → Week → Day
- City → Store → Terminal
- Adição de dimensões

Slice

Seleciona uma única fatia do cubo fixando uma dimensão em um valor específico.

Exemplo: vendas apenas de Janeiro (time = "Jan")

Dice

Seleciona um subcubo definindo condições em múltiplas dimensões simultaneamente.

Exemplo: TVs e Monitores, Q1 e Q2, Brasil e Argentina

Pivot (Rotate)

Rotaciona o cubo para visualizar dados de uma perspectiva diferente, reorganizando dimensões nos eixos.

❏ **Caso de Uso:** Um executivo pode começar visualizando vendas mensais totais por cidade (roll-up), identificar uma anomalia, e então fazer drill-down para examinar vendas diárias por loja específica e categoria de produto para investigar a causa raiz.

Implementação OLAP

A implementação de sistemas OLAP (Online Analytical Processing) representa um marco fundamental na construção de soluções modernas de data warehousing. Esta apresentação explora as principais estratégias, técnicas e arquiteturas necessárias para implementar cubos OLAP eficientes e escaláveis em ambientes corporativos complexos.

Exemplo de Consulta OLAP

Cenário de Análise

Tabela de ensaios médicos com atributos como idade, gênero, sucesso do teste, entre outros. Objetivo: encontrar o número total de ensaios bem-sucedidos.

Métodos Tradicionais

- **Método 1:** Varredura completa da tabela uma única vez
- **Método 2:** Construção de índice B+ tree no atributo "sucesso", ainda requerendo acesso a todos os registros

Desafio Central

É possível obter a contagem sem escanear todos os registros? Isso motiva técnicas de pré-computação OLAP.

Consulta OLAP com Cubo de Dados
Referência: Han, Kamber & Pei (2011)

ENTRADA: T (tabela de ensaios médicos)

atributos: idade, genero, sucesso, ...

PRÉ-CONDIÇÃO: T carregada no Data Warehouse

PASSO 1 — Definição do Cubo

COMPUTE CUBE ensaios AS

SELECT idade, genero, sucesso, COUNT(*)

FROM T

CUBE BY idade, genero, sucesso

PASSO 2 — Materialização Seletiva

PARA CADA cuboide C em lattice(ensaios):

SE relevante(C): # ex: envolve 'sucesso'

materializar(C) # pré-computar e armazenar

PASSO 3 — Consulta Eficiente

Sem varredura completa da tabela:

resultado ← buscar_cuboide(sucesso = TRUE)

RETORNAR COUNT(*) de resultado

SAÍDA: total de ensaios bem-sucedidos

PÓS-CONDIÇÃO: resposta obtida via lookup,
sem escanear registros individuais

Exemplo de Join Index

Conceito de Join Index

Estrutura de dados especializada que pré-computa e armazena resultados de operações de junção entre tabelas, eliminando joins repetidos em consultas analíticas.

Vantagens

- Redução drástica no tempo de resposta
- Minimização de I/O em disco
- Otimização para queries complexas

Aplicação em OLAP

Especialmente útil para conectar tabelas fato com tabelas dimensão, acelerando consultas que envolvem múltiplas dimensões.

Algoritmo de Construção de Join Index
Referência: Valduriez (1987); Han et al. (2011)

ENTRADA: F (tabela fato), D[] (tabelas dimensão), chaves_join[] (atributos de junção)

PRÉ-CONDIÇÃO: F e D[] carregadas no Data Warehouse

PASSO 1 — Construção do Join Index

PARA CADA dimensão D_i em $D[]$:

PARA CADA tupla t em F :

$rid_f \leftarrow \text{rowid}(t)$

$rid_di \leftarrow \text{buscar}(D_i, \text{chaves_join}[i], t.\text{chave})$

$\text{join_index}[D_i] \leftarrow \text{join_index}[D_i] \cup (rid_f, rid_di)$

PASSO 2 — Armazenamento

PARA CADA $\text{join_index}[D_i]$:

$\text{ordenar}(\text{join_index}[D_i], \text{por } rid_f)$

$\text{persistir}(\text{join_index}[D_i])$ # índice materializado

PASSO 3 — Uso em Consulta Analítica

DADO query Q envolvendo F D_1 D_2 :

$rids_f \leftarrow \text{join_index}[D_1].\text{lookup}(\text{filtro_D1})$

$\cap \text{join_index}[D_2].\text{lookup}(\text{filtro_D2})$

$\text{resultado} \leftarrow \text{fetch}(F, rids_f)$ # acesso direto

RETORNAR $\text{agregar}(\text{resultado}, Q.\text{medidas})$

SAÍDA: resultado da consulta sem recomputar joins

PÓS-CONDIÇÃO: tempo de resposta $O(|\text{resultado}|)$

em vez de $O(|F| \times |D_1| \times |D_2|)$

Armazenamento de Dados: Horizontal vs. Colunar

A escolha da estratégia de armazenamento físico tem impacto profundo no desempenho de sistemas OLAP. Tabelas fato frequentemente são "gordas", com dezenas ou centenas de dimensões e atributos.

Armazenamento Horizontal

Tuplas armazenadas sequencialmente, uma após a outra. Cada linha contém todos os atributos do registro.

Armazenamento Vertical (Colunar)

Tuplas armazenadas por atributos, com cada coluna mantida separadamente em estruturas otimizadas.

❏ **Insight Crítico:** Consultas analíticas típicas acessam apenas alguns atributos de tabelas muito largas. O armazenamento colunar otimiza esse padrão, lendo apenas as colunas necessárias.

Vantagens do Armazenamento Colunar

Compressão Eficiente

Valores similares ficam agrupados, permitindo taxas de compressão superiores ao modelo row-based.

I/O Reduzido

Apenas as colunas relevantes são lidas do disco, eliminando dados desnecessários no processamento.

Processamento Vetorizado

Operações executadas em blocos de dados contíguos, aproveitando o cache de CPU com maior eficiência.

Benefícios em OLAP: aceleração de consultas agregadas (SUM, AVG, COUNT), menor consumo de memória, melhor desempenho em queries seletivas e escalabilidade para tabelas muito largas.

Sistemas modernos: *Amazon Redshift*, *Google BigQuery* e *Apache Parquet* utilizam esta abordagem como base de sua arquitetura de armazenamento.

Arquiteturas de Servidores OLAP

A escolha da arquitetura de servidor OLAP determina o equilíbrio entre desempenho, escalabilidade e flexibilidade. Cada abordagem oferece trade-offs específicos para diferentes casos de uso corporativos.

ROLAP (Relational OLAP)

Utiliza SGBD relacional ou relacional estendido para armazenar e gerenciar dados do warehouse, com middleware OLAP intermediário. Inclui otimização de backend, lógica de navegação de agregações e ferramentas adicionais.

Vantagem: Maior escalabilidade para grandes volumes.

MOLAP (Multidimensional OLAP)

Motor de armazenamento multidimensional baseado em arrays esparsos. Oferece indexação rápida para dados sumarizados pré-computados.

Vantagem: Desempenho superior em consultas complexas.

HOLAP (Hybrid OLAP)

Combina flexibilidade de abordagens relacionais e multidimensionais. Exemplo: Microsoft SQL Server. Baixo nível usa modelo relacional; alto nível emprega estruturas de array.

Vantagem: Equilíbrio entre desempenho e escalabilidade.

Servidores SQL Especializados

Exemplos incluem Redbricks e outros sistemas otimizados. Suporte especializado para consultas SQL sobre esquemas star/snowflake. **Vantagem:** Otimizações específicas para workloads analíticos.

Materialização de Cubos: Full Cube vs. Iceberg Cube

Comparação de Abordagens

A materialização completa de cubos pode ser computacionalmente proibitiva. O Iceberg Cube computa apenas células que satisfazem condições específicas.

Eficiência de Espaço

Apenas uma pequena porção de células pode estar "acima da água" em um cubo esparsos

Redução de Computação

Evita processar células com valores insignificantes

Focado em Insights

Concentra recursos em padrões estatisticamente relevantes

❏ **Condição Iceberg:** Computa apenas células cuja medida satisfaz o critério definido (ex: COUNT(*) >= 100).

Algoritmo Iceberg Cube
Referência: Beyer & Ramakrishnan (1999)

ENTRADA: R (relação base), dims[] (dimensões),
 medida (ex: COUNT), min_sup (limiar)
PRÉ-CONDIÇÃO: min_sup > 0

PASSO 1 — Geração do Cubo Iceberg
COMPUTE CUBE vendas ICEBERG AS
SELECT mes, cidade, grupo_cliente, COUNT(*)
FROM salesInfo
CUBE BY mes, cidade, grupo_cliente
HAVING COUNT(*) >= min_sup

PASSO 2 — Poda Antimonotônica
PARA CADA cuboide C na lattice (ordem top-down):
PARA CADA célula c em C:
SE medida(c) < min_sup:
 podar(c) # c e todos descendentes
 CONTINUAR # não descer na lattice

PASSO 3 — Materialização Seletiva
PARA CADA célula c não podada:
 armazenar(c, medida(c))

SAÍDA: subconjunto de células "acima da linha d'água"
PÓS-CONDIÇÃO: apenas células com medida >= min_sup
 são materializadas e armazenadas

OLAP de Alta Dimensionalidade: A Maldição da Dimensionalidade

Cenários de Alta Dimensionalidade

- **Análise científica e de engenharia:** Simulações complexas com centenas de variáveis
- **Bio-data analysis:** Milhares de genes e marcadores biológicos
- **Pesquisas estatísticas:** Centenas de variáveis demográficas e comportamentais

Exemplo Ilustrativo

Base de dados com 600.000 tuplas. Cada dimensão tem cardinalidade de 100 e distribuição Zipf de 2. O espaço de possibilidades explode exponencialmente.

Limitações dos Métodos Tradicionais

Iceberg e Cubos Comprimidos

Apenas atrasam a explosão combinatória inevitável

Materialização Completa

Overhead significativo no acesso a resultados em disco

- ❑ Nenhum método anterior de cubing consegue lidar adequadamente com alta dimensionalidade. Novas abordagens algorítmicas são essenciais.

Algoritmo Shell-Fragment Cubing

Referência: Li, Han & Gonzalez (VLDB'04)

ENTRADA: R (relação base), $\text{dims}[]$ (d dimensões),
 k (tamanho máximo do fragmento shell)

PRÉ-CONDIÇÃO: $d \gg k$ (alta dimensionalidade)

PASSO 1 — Particionamento em Shell-Fragments

PARA $i = 1$ ATÉ d PASSO k :

$\text{frag} \leftarrow \text{dims}[i .. i+k-1]$ # fragmento de k dims

$\text{shell_fragments} \leftarrow \text{shell_fragments} \cup \{\text{frag}\}$

PASSO 2 — Cubing Local por Fragmento

PARA CADA frag em shell_fragments :

$\text{cubo_local} \leftarrow \text{computar_cubo}(R, \text{frag})$

$\text{materializar}(\text{cubo_local})$ # armazenar em memória

PASSO 3 — Resposta a Consultas por Combinação

DADO query Q sobre $\text{dims}_Q \subseteq \text{dims}[]$:

$\text{fragmentos_relevantes} \leftarrow \{f \in \text{shell_fragments} \mid$
 $\text{dims}_Q \cap f \neq \emptyset\}$

$\text{resultado} \leftarrow \text{combinar}(\text{fragmentos_relevantes}, Q)$

RETORNAR resultado

SAÍDA: cubos locais materializados por fragmento

PÓS-CONDIÇÃO: consultas respondidas sem materializar
o cubo completo de d dimensões

Referências Principais

Esta seleção de referências representa os pilares fundamentais para o estudo aprofundado de mineração de dados, cobrindo desde conceitos básicos até técnicas avançadas e aplicações contemporâneas.



1. **J. Han, J. Pei, and H. Tong**, *Data Mining: Concepts and Techniques*, 4th edition. Cambridge, MA: Morgan Kaufmann, 2022.
2. **G. M. James, D. Witten, T. Hastie, and R. Tibshirani**, *An Introduction to Statistical Learning: With Applications in R*. Springer Nature, 2021.
3. **S. Garcia, J. Luengo, and F. Herrera**, *Data Preprocessing in Data Mining*. Springer, 2014.
4. **C. M. Bishop and H. Bishop**, *Deep Learning: Foundations and Concepts*. Springer Nature, 2023.